COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CpE 010 - Data Structures and Algorithms	**SECTION:**	CpE21S1
DATE PERFORMED:	7-16-2026	**DATE SUBMITTED:**	7-16-2026
NAME:	Mauro, Ryan Lewis B. Mauro	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 [ARRAYS, POINTERS, AND DYNAMIC MEMORY ALLOCATION]

1. PROCEDURE OUTPUT

ILO A:

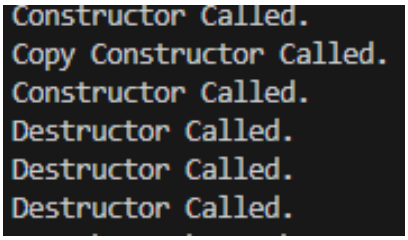
Screenshot	
Observation	The **constructor** was called when creating **student1** and **student3**, while the **copy constructor** was called when creating **student2** from **student1**. The **assignment student3 = student2** used the default **copy assignment operator** provided by C++, so no message was displayed. When the program ended, the **destructor** was called for all three Student objects.

Table 2-1. Initial Driver Program

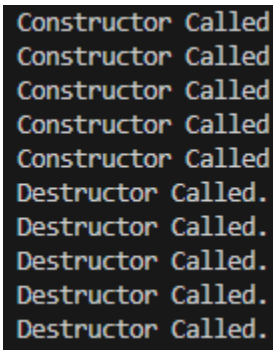
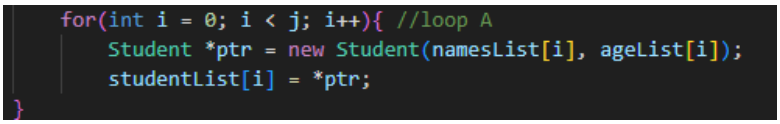
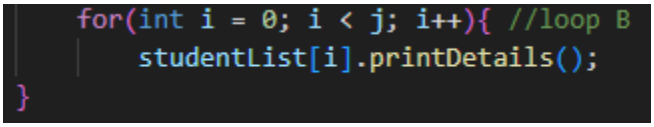
Screenshot	
Observation	The program creates an array of five Student objects using the **default constructor**, so the **constructor is called five times**. The **namesList** and **ageList arrays** are only declared and are not yet used to assign values to the students. When the program ends, the **destructor** is called for each of the five objects.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	Loop A dynamically creates a new Student object using the new keyword with the name and age from the arrays. The object is then copied into the studentList array. Since delete ptr; is not used, the dynamically allocated memory is not released, which may cause a memory leak.
Loob B	
Observation	Loop B displays the details of each Student object stored in the studentList array by calling the printDetails() function. It prints the name and age of all five students.

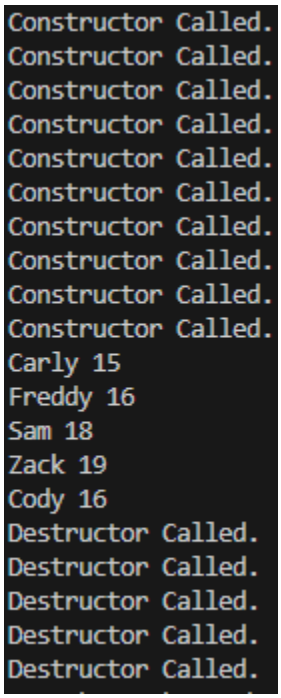
Output	
Observation	The program successfully **creates five Student objects dynamically**, copies their data into the **studentList array**, and **displays the names and ages of the students**. At the end of the program, only the **destructors** for the objects in **studentList** are called. The dynamically allocated objects are not deleted because delete ptr; is missing, resulting in a memory leak.

Table 2-3. Final Driver Program

Modification	`delete ptr;  // modification line`
Observation	After adding **delete ptr**;, the dynamically allocated Student object is **deleted immediately** after its data is copied into the studentList array. This **releases the allocated memory and prevents memory leaks**. The program still displays the same student information, but memory is managed correctly because each dynamically created object is properly destroyed after use.

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

```
11  public:
12      // Problem 1 Constructor
13      GroceryItem(std::string n = "", double p = 0, int q = 0)
14      {
15          name = n;
16          price = p;
17          quantity = q;
18      }
19
20      // Problem 1 Destructor
21      virtual ~GroceryItem() {}
22
23      // Problem 1 Copy Constructor
24      GroceryItem(const GroceryItem &other)
25      {
26          name = other.name;
27          price = other.price;
28          quantity = other.quantity;
29      }
30
31      // Problem 1 Copy Assignment Operator
32      GroceryItem &operator=(const GroceryItem &other)
33      {
34          if (this != &other)
35          {
36              name = other.name;
37              price = other.price;
38              quantity = other.quantity;
39          }
40          return *this;
41      }
```

Figure 3.1 <Supplementary code for Prob. 1>

The **constructor** initializes the grocery item's name, price, and quantity when a new object is created. The **destructor** is automatically called when the object is no longer needed. The **copy constructor** creates a new object by copying the data from another object, while the **copy assignment operator** copies the data from one existing object to another existing object.

```
45      // Problem 1 Function to calculate total
46      double calculateSum() const
47      {
48          return price * quantity;
49      }
50
51      // Problem 1 Function to display item details
52      void display()
53      {
54          std::cout << "Type: " << getType() << std::endl;
55          std::cout << "Name: " << name << std::endl;
56          std::cout << "Price: PHP " << price << std::endl;
57          std::cout << "Quantity: " << quantity << std::endl;
58          std::cout << "Item Total: PHP " << calculateSum() << std::endl;
59      }
```

Figure 3.2 <Supplementary code for Prob. 1>

The **calculateSum()** function computes the total cost of an item by multiplying its price and quantity. The **display()** function prints the item's type, name, price, quantity, and total cost so the user can easily view all of its details.

```cpp
// Problem 1 Fruit Class
class Fruit : public GroceryItem
{
public:
    Fruit(std::string n, double p, int q)
        : GroceryItem(n, p, q) {}

    ~Fruit() {}

    Fruit(const Fruit &other)
        : GroceryItem(other) {}

    Fruit &operator=(const Fruit &other)
    {
        GroceryItem::operator=(other);
        return *this;
    }

    std::string getType()
    {
        return "Fruit";
    }
};
```

Figure 3.3 <Supplementary code for Prob. 1>

It creates a specific category for fruits, gives it tools to set up its initial information (name, price, and quantity), allows it to safely copy or assign fruit data, and includes a function that simply identifies itself by returning the word "Fruit".

```cpp
// Problem 1 Vegetable Class
class Vegetable : public GroceryItem
{
public:
    Vegetable(std::string n, double p, int q)
        : GroceryItem(n, p, q) {}

    ~Vegetable() {}

    Vegetable(const Vegetable &other)
        : GroceryItem(other) {}

    Vegetable &operator=(const Vegetable &other)
    {
        GroceryItem::operator=(other);
        return *this;
    }

    std::string getType()
    {
        return "Vegetable";
    }
};
```

Figure 3.4 <Supplementary code for Prob. 1>

The Vegetable class inherits from GroceryItem, allowing it to reuse common grocery item features such as name, price, and quantity. Its **constructor** initializes these values through the parent class, while the **copy constructor** and **assignment operator** ensure objects can be copied correctly. The **getType()** function returns "Vegetable" to identify the item's category.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

```
// Problem 2 Create GroceryList array using dynamic memory
GroceryItem *GroceryList[SIZE];

GroceryList[0] = new Fruit("Apple", 10, 7);
GroceryList[1] = new Fruit("Banana", 10, 8);
GroceryList[2] = new Vegetable("Broccoli", 60, 12);
GroceryList[3] = new Vegetable("Lettuce", 50, 10);

// Problem 2 Display all grocery items
std::cout << "JENNA'S GROCERY LIST\n\n";

for (int i = 0; i < SIZE; i++)
{
    GroceryList[i]->display();
    std::cout << "-------------------------\n";
}
```

Figure 3.5 <Supplementary code for Prob. 2>

This code creates a dynamic array of GroceryItem pointers and stores Fruit and Vegetable objects in it. It then displays the heading **"Jenna's grocery list"** and uses a for loop to call the **display()** function for each object, demonstrating polymorphism by displaying the correct details for each grocery item.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

```
// Problem 3 Function to calculate total price of all items
double TotalSum(GroceryItem *list[], int size)
{
    double total = 0;

    for (int i = 0; i < size; i++)
    {
        if (list[i] != nullptr)
        {
            total += list[i]->calculateSum();
        }
    }

    return total;
}
```

Figure 3.6 <Supplementary code for Prob. 3>

TotalSum() function, which calculates the total price of all grocery items by summing the results of each object's calculateSum() function.

```
// Problem 3Display total amount
std::cout << "\nTotal Amount to Pay: PHP "
          << TotalSum(GroceryList, SIZE)
          << std::endl;
```

Figure 3.7 <Supplementary code for Prob. 3>

This code displays the total amount the customer needs to pay by calling the **TotalSum()** function, which calculates the total cost of all the grocery items in the GroceryList array. After showing the total amount with the label "**Total Amount to Pay: PHP**", it moves the cursor to the next line using **std::endl.**

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

```
// Problem 4 Delete Lettuce and dde-allocate memory
std::cout << "\nDeleting Lettuce...\n";

delete GroceryList[3];
GroceryList[3] = nullptr;

std::cout << "Updated Total Amount: PHP "
          << TotalSum(GroceryList, SIZE)
          << std::endl;

for (int i = 0; i < SIZE; i++)
{
    delete GroceryList[i];
    GroceryList[i] = nullptr;
}

return 0;
```

Figure 3.8 <Supplementary code for Prob. 4>

This code deletes the lettuce item from the grocery list by freeing its allocated memory and setting its pointer to **nullptr** to avoid a dangling pointer. It then recalculates and displays the updated total amount after the deletion. Finally, it deletes the remaining grocery items and sets their pointers to **nullptr** to properly release all dynamically allocated memory before the program ends.

Output:

```
Type: Fruit
Name: Apple
Price: PHP 10
Quantity: 7
Item Total: PHP 70
-------------------------
Type: Fruit
Name: Banana
Price: PHP 10
Quantity: 8
Item Total: PHP 80
-------------------------
Type: Vegetable
Name: Broccoli
Price: PHP 60
Quantity: 12
Item Total: PHP 720
-------------------------
Type: Vegetable
Name: Lettuce
Price: PHP 50
Quantity: 10
Item Total: PHP 500
-------------------------

Total Amount to Pay: PHP 1370

Deleting Lettuce...
Updated Total Amount: PHP 870
```

Figure 3.9 <Output for Supplementary code>

The program displays the details of each grocery item, including its type, name, price, quantity, and the total cost for each item. It then calculates the total amount of all the groceries, which is PHP 1,370. After that, the program removes the Lettuce item from the grocery list, recalculates the total without it, and displays the updated amount of PHP 870. Before the program ends, it frees the memory used by all the grocery items to ensure proper memory management.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

- **I used it for grammar correction and asked why my code still encountered errors and it turns out I have missing colons and wrong spelling for my variables.**
- **Asked chatgpt how to use dynamic memory for array and how to use polymorphism for displaying a list**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.